

AssemblyAI ↔ YouTube Sync Analysis

Date: May 26, 2026

Branch: `feature/v153-truly-clean`

Author: Abacus AI Agent (investigating for Larry)

The Big Question

Was AssemblyAI ever successfully synced with YouTube video playback?

Answer: NO — Conclusively, it was never achieved.

The evidence is overwhelming. AssemblyAI timestamps were **always NaN**, the backend API route was **never created**, and every version of the sync code eventually **abandoned real timestamps** in favor of synthetic timing estimates. A periodic sync interval was built (v80), then **removed** (v122). The legacy integration accumulated 2,550+ lines of workaround code trying to solve a problem that was structurally impossible given the architecture.

Evidence Summary

1. The Backend Route Never Existed

`useChunkedTranscription.ts` (lines 168-169) calls `/api/assemblyai-transcription` and divides response timestamps by 1000 (ms → seconds). **But this API route was never created.** Without a backend, the hook returned empty/malformed segments, meaning all downstream timestamp fields were `undefined` or `NaN`.

2. Timestamps Were Always NaN

From `useAudioClarification.tsx` (legacy, v129, 2,550 lines):

Version	Line	Evidence
v103	~399	"v103 bug: raw segments have NaN timing, so target-Duration was always undefined"
v85	~346	"Final NaN guard - NEVER return NaN" — a last-resort defensive check
v91	~338	"Previously used word-count-based: (wordCount / 150) * 60 which created ~3s segments with ~15s spacing → HUGE GAPS"

3. resolveSegmentTiming: 6 Fallback Paths (All Failed)

The function at line 279 of `useAudioClarification.tsx` is ~70 lines long. It tries **six different ways** to extract a valid start time from a segment:

1. `seg.start` (direct)
2. `raw.startMs / 1000` (millisecond conversion)
3. `raw.startTime` (alternative field name)
4. `raw.start` (yet another field name)
5. `raw.words?.[0]?.start / 1000` (first word's start time)
6. **Fallback:** `index * avgDuration` (evenly distributed — NOT synced at all)

Having six fallback paths means the data was **never in a consistent format**. The final fallback (evenly distributing segments across video duration) proves that real timestamps were never reliably available.

4. adaptSegmentsForAudio: Timestamps Thrown Away

The `adaptSegmentsForAudio` function (line 404) completely **abandons original AssemblyAI timestamps** and creates synthetic timing:

```
totalWords = sum of all segment word counts
avgDuration = videoDuration / totalSegments
each segment.startTime = index * avgDuration
each segment.endTime = (index + 1) * avgDuration
```

This is estimation, not synchronization. The segments are evenly spaced regardless of when they actually occur in the video.

5. Periodic Sync Was Built and Removed

Lines 899-908 of `useAudioClarification.tsx` :

```
// v122: REMOVED isPlayingRef (was only used by periodic sync interval, now removed)
// v122: REMOVED syncIntervalRef (periodic sync interval removed - v80 handles trans-
itions)
// v122: REMOVED currentlyPlayingSegmentIndexRef (periodic sync removed)
// v122: REMOVED avgSegmentDurationRef (periodic sync removed)
// v122: REMOVED adaptedSegmentsRef (periodic sync removed)
```

A sync interval (checking video time and adjusting audio playback) was implemented around v80 and **completely removed** in v122. The replacement was `playAtTime()` calls — which just play the nearest TTS segment when the video reaches a certain time, with no drift correction.

Why Sync Was Structurally Impossible

Even if AssemblyAI timestamps had been valid, there's a **fundamental mismatch**:

Source	Timestamps Reference
AssemblyAI	The extracted audio file (starts at 0:00 of the audio stream)
YouTube Player	The video timeline (may include ads, intros, buffering delays)
YouTube Captions	The video timeline (already aligned by YouTube)

The extracted audio and the video player timeline are **not guaranteed to be aligned**:

- Audio extraction via `yt-dlp` may start at a slightly different point
- YouTube's player may have buffering or ad-related offsets
- There's no API to query the exact offset between the two

This means AssemblyAI timestamps would require an **unknown offset correction** to match the video timeline — a problem that was never solved.

The Git History Tells the Story

Sync-related commits show a long struggle:

Commit	Message
d4bf58f	feat: Implement Clarify Audio with George's sync algorithm
cb03900	Implement George synchronization algorithm (v12)
51e92d7	v14: TRUE George algorithm with automatic sync
5e9629b	v19: Subtitle-only version - remove all audio, focus on perfect subtitle sync
ab0ab3d	Implement elastic sync - no more repetition
2a623be	Fix elastic sync monitoring loop
81fb3a7	Add two-tier sync: jump for large drift, elastic for small
71b379f	Implement train sync - one global rate, one jump, smooth sailing
166f78c	Fix train sync: correct global rate calc, handle user speed
f5ae93c	Implement scheduler approach - natural speed AI, timing-based sync
9bdcd0f	CRITICAL FIX: Race condition – syncAudio used stale audioMode closure
b631c0e	v153: Three sync fixes – expose player, real timestamps, pass currentTime

That's **12+ iterations** of sync algorithms across the project's history. The approach was reinvented multiple times because none of them fully worked with AssemblyAI's timestamp model.

How the Current System Avoids This Problem

The active `ClarifyAudioPanel.tsx` (~1,559 lines) uses a completely different approach:

YouTube Captions → TTS → Scheduler

1. **Captions from YouTube API** — timestamps are already aligned to the video timeline
2. **TTS generation per segment** — each caption segment gets its own audio blob
3. **Scheduler loop** (line 479) — a `setInterval` that checks `currentTime` from the video player

4. **Segment selection** (line 808, `findSegForTime`) — binary search: `if (videoTime >= segs[i].start)`

5. **Play when ready** — if the target segment has generated audio, play it

Why this works: YouTube captions are authored against the video timeline. There's no offset to correct. When the video player says "we're at 45.2 seconds," the caption that starts at 45.0s is the right one, and its TTS audio plays immediately.

“GEORGE THE VIDEO EDITOR” (`useAudioTranslation.ts`)

The legacy `useAudioTranslation.ts` (4,232 lines) developed an elaborate sync system called “George” (v97+):

- **500ms monitoring loop** checking drift between audio position and video position
- **Playback rate adjustment** (0.8x–1.3x) to gradually reduce drift
- **Jump threshold** (2.5s) — if drift exceeds this, jump to correct segment
- **Cooldown** (2000ms) — prevent thrashing from rapid jumps

George works because it operates on **YouTube caption timestamps** (already aligned). It would NOT have worked with AssemblyAI timestamps without the offset correction that was never implemented.

Recommended Approach: Hybrid (If AssemblyAI Is Pursued)

Based on this analysis, the recommended path forward is:

Use AssemblyAI for Speaker Labels Only

YouTube Captions (already synced) → text + timestamps
 AssemblyAI Transcription → speaker labels (Speaker A, Speaker B)
 Match by text similarity → **assign** speaker labels to YouTube caption segments

Why this works:

- YouTube timestamps handle sync (proven working)
- AssemblyAI handles speaker detection (proven accurate — see `ASSEMBLYAI_INVESTIGATION.md`)
- Text matching bridges the two without needing timestamp alignment
- Cost is still ~\$0.04/video (one AssemblyAI call per video)

Implementation Sketch

1. Call AssemblyAI with `speaker_labels=true` on extracted audio
2. Get back utterances with `speaker` field + `text` field
3. For each YouTube caption segment, find the AssemblyAI utterance with the most similar text (fuzzy match)
4. Assign that utterance's `speaker` label to the caption segment
5. Use existing `ClarifyAudioPanel` scheduler for playback (no changes needed)

This completely **sidesteps the sync problem** that defeated the legacy integration.

Conclusion

Question	Answer
Was sync ever achieved?	No
Why not?	Backend API route never existed; timestamps were always NaN; fundamental timeline mismatch between AssemblyAI and YouTube
How many attempts?	12+ sync algorithm iterations across project history
What finally worked?	Abandoning AssemblyAI timestamps entirely; using YouTube captions (already aligned)
Should we use AssemblyAI?	Yes, but only for speaker labels — let YouTube handle timing
Estimated cost?	~\$0.04/video (unchanged)